

Module : ALGORITHME

Leçon 3 : Les sous-programmes

Introduction

Dans le développement logiciel, la taille et la complexité d'un programme peuvent rapidement devenir difficiles à gérer. Pour organiser le code, le rendre réutilisable et plus lisible, on utilise les **sous-programmes**.

Un **sous-programme** est un bloc d'instructions que l'on peut exécuter indépendamment, à plusieurs endroits dans le programme, sans avoir à réécrire le code.

Les sous-programmes peuvent prendre **des entrées** (paramètres), effectuer des traitements et parfois **renvoyer un résultat**.

1. Définition et types de sous-programmes

On distingue généralement deux types de sous-programmes :

a) Les **procédures**

- Une **procédure** exécute des instructions mais ne renvoie pas de valeur.
- Elle sert surtout pour **répéter une action**.
- **Exemple en pseudo-code :**

Procédure AfficherMessage

Afficher "Bonjour !"

FinProcédure

- On peut l'appeler plusieurs fois :

AfficherMessage

AfficherMessage

b) Les **fonctions**

- Une **fonction** renvoie une **valeur** après traitement.
- Elle prend éventuellement des paramètres et retourne un résultat.
- **Exemple en pseudo-code :**

Fonction Additionner(a, b)

Retourner a + b

FinFonction

- On peut l'utiliser dans une expression :

`x ← Additionner(5, 3) // x vaut 8`

2. Pourquoi utiliser des sous-programmes ?

1. **Réutilisabilité** : écrire une fois, utiliser plusieurs fois.
 2. **Lisibilité** : un programme bien structuré est plus facile à comprendre.
 3. **Maintenance** : corriger une erreur dans un sous-programme corrige tous les appels.
 4. **Décomposition** : diviser un problème complexe en sous-problèmes plus simples.
-

3. Paramètres

Les sous-programmes peuvent recevoir des **paramètres** : ce sont les données qu'ils utilisent pour effectuer leurs tâches.

a) Paramètres **d'entrée**

- Fournissent des informations au sous-programme.
- Exemple : `Additionner(a, b)` prend a et b comme paramètres.

b) Paramètres **de sortie**

- Permettent de retourner des valeurs ou des résultats multiples.
- Dans certains langages, on utilise des mots-clés comme `out` ou `réf.`

c) Paramètres **modifiables**

- Certains langages permettent au sous-programme de **modifier directement la variable passée en paramètre**.
-

4. Appel d'un sous-programme

Pour utiliser un sous-programme, il faut **l'appeler** dans le programme principal.

Syntaxe générale :

`NomSousProgramme(paramètres)`

Exemple :

`Procédure DireBonjour(nom)`

`Afficher "Bonjour, " + nom + " !"`

`FinProcédure`

`DireBonjour("Lamine") // Affiche : Bonjour, Lamine !`

5. Exemple complet

Voici un exemple d'un petit programme avec sous-programmes :

Pseudo-code :

Fonction Carré(nombre)

 Retourner nombre * nombre

FinFonction

Procédure AfficherRésultat(x)

 Afficher "Le carré est : " + x

FinProcédure

// Programme principal

n ← 5

résultat ← Carré(n)

AfficherRésultat(résultat)

Explications :

1. La fonction Carré calcule le carré d'un nombre.
2. La procédure AfficherRésultat affiche le résultat.
3. Le programme principal utilise ces sous-programmes pour éviter de répéter du code.

6. Bonnes pratiques

- Donner un **nom clair** aux sous-programmes.
- Limiter le nombre de paramètres (idéal : ≤ 3).
- Commenter la fonction ou procédure pour expliquer son rôle.
- Éviter les effets de bord (modification inattendue des variables globales).

7. Exercices

1. Écrire une **fonction** qui retourne le double d'un nombre.
2. Écrire une **procédure** qui affiche "Bienvenue à [nom]" et tester avec plusieurs noms.
3. Créer une **fonction** qui calcule la moyenne de trois nombres et une procédure qui affiche cette moyenne.